

AI-POWERED

Computer Vision & Audio Intelligence Suite

Comprehensive Technical Project Report

Six Integrated Machine Learning Applications

Internship Report for ELEVANCE

Date: 25-04-2026 to 25-05-2026

Internship Name: Learn To Build A Real Time Application For Gender And Age Detector

Project Repo: [All Projects Repository](#)

Executive Summary

This report documents a suite of six end-to-end machine learning applications developed using Python, Streamlit, PyTorch, and the YOLOv8 family of object detection models. Each application solves a distinct real-world AI problem — from voice-based age and emotion detection, to multi-model traffic scene analysis, long-hair-based gender inference, nationality identification, senior citizen tracking, and sign language recognition.

All six projects follow a consistent pipeline: dataset acquisition, data preparation and labelling, model training or integration of pre-trained transformer-based models, conditional inference logic, and a Streamlit-powered GUI. Models are stored on Google Drive and downloaded at runtime via gdown, ensuring portability across environments.

Suite Overview

#	Project	Input Type	Key Output	Deployment
1	Voice Age & Emotion Detection	Audio (.wav/.mp3)	Age, Gender, Emotion	Streamlit Web App
2	Car Colour & Traffic Intelligence	Image	Car count, Blue cars, Pedestrians	Streamlit Web App
3	Long Hair & Conditional Gender	Face Image	Hair length, Age group, Gender	Streamlit Web App
4	Nationality & Attribute ID	Person Image	Nationality, Age, Emotion, Dress Colour	Streamlit Web App
5	Senior Citizen Video Analysis	Video / Webcam	Age, Gender, Senior flag, CSV log	Streamlit Web App
6	ASL Sign Language Recognition	Image / Video / Live	Sign letter or word	Streamlit Web App

Project 1: Voice Age & Emotion Detection

Problem Statement

Develop a machine learning model to detect a person's age from a voice note. The model must process only male voices — if a female voice is detected, it should reject the input with the message "Upload male voice." For individuals over 60, the system marks them as senior citizens and additionally detects their emotion. For those below 60, only age is reported.

Dataset

Dataset	Common Voice Dataset v4 (Kaggle: vedant2022/common-voice-dataset-version-4)
Size	~5,000 balanced audio clips sampled across age categories
Sampling Strategy	Up to 700 clips per age group; groups with fewer than 700 use all available samples
Format	TSV metadata (train.tsv) + MP3/WAV audio clips
Labels Used	age, gender columns from TSV

Model Architecture

Three specialised models are used in a tiered pipeline:

Model	Role	Architecture	Source
Gender Classifier	Gate — rejects female voices	Audio classification pipeline	HuggingFace (gender_model)
Age Estimator	Predicts speaker age (0-100 scale)	Wav2Vec2 base + custom regression head (ModelHead)	Fine-tuned: age_model (Google Drive)
Emotion Classifier	Detects emotion for 60+ year-olds	Audio classification pipeline	HuggingFace (emotion_model)

Technical Details

Custom Model (InferenceWrapper)

A custom PyTorch nn.Module wraps the Wav2Vec2 base model with two ModelHead sub-modules: one for age regression (output size 1) and one for gender classification (output size 3, softmax-activated). The model extracts last_hidden_state from Wav2Vec2, mean-pools across the time dimension, and passes the result through each head independently. All weights are forced to float32 for CPU compatibility.

Inference Flow

- Audio is loaded with librosa at 16 kHz.

- The gender pipeline runs first; female voice triggers immediate rejection.
- For male voices, inputs are processed by Wav2Vec2Processor and fed to InferenceWrapper.
- Age is extracted from the regression logit multiplied by 100.
- If age > 60, the emotion pipeline is called and the result is displayed alongside the senior citizen flag.

Key Logic

The system implements a three-stage conditional gate: (1) gender check, (2) age threshold, (3) conditional emotion detection. This design avoids unnecessary inference calls and enforces the problem constraints strictly.

GUI Features

- Single-file audio uploader accepting WAV and MP3 formats.
- Audio playback widget for the uploaded file.
- Results displayed using st.success, st.info, and st.error for clear visual feedback.
- All models loaded once via @st.cache_resource, preventing redundant downloads.

Project 2: Car Colour & Traffic Scene Intelligence

Problem Statement

Develop a model to count cars in traffic images, identify blue cars (marked with a red bounding box), detect traffic signals to classify the scene, and — only for traffic signal scenes — count the number of pedestrians present. Non-traffic-signal scenes skip people detection entirely.

Dataset

Dataset	Indian Driving Dataset (Kaggle: redzapos123/indian-driving-dataset-detections-yolov11)
Size	2,500 curated images containing cars, traffic lights, and/or persons
Format	YOLOv8 format — images + .txt label files + data.yaml
Classes Used	car, traffic light, person (dynamically discovered via model.names dictionary)

Model Ensemble

Model	Alias	Purpose
YOLOv2.6n	yolo26n	Fast pedestrian detection
YOLOv2.6s	yolo26s	Medium pedestrian detection
YOLOv2.6x	yolo26x	High-accuracy signal + pedestrian detection
YOLOv8n (IDD-trained)	idd_v8	Indian road scene person detection
YOLOv8x-worldv2	lvis_v8	Open-world person detection
YOLOv8x-oiv7	car_expert	Car detection specialist

Technical Details

Dynamic Class ID Discovery

Rather than hard-coding class IDs, the system iterates over each model's .names dictionary and matches keyword lists (person/pedestrian/rider, car/vehicle, traffic light/signal). This makes the pipeline robust to any YOLO model variant regardless of training-set ordering.

Car Detection (Step 1)

A quad-split strategy runs the car expert model on each image quadrant independently to catch small cars near edges, then runs it on the full image for boundary-crossing vehicles. Duplicate suppression uses a custom IoU + nested-box check (IoU > 0.4 OR one box is 85% inside the other). The final car count is the maximum of the whole-image count and the quadrant sum plus boundary boxes.

Blue Car Detection

A channel-based rule in RGB space flags pixels where $B > R$, $B > G$, $B > 50$, and $B > (R + G) * 0.65$. A crop is considered a blue car if more than 15% of its pixels satisfy this mask. Blue cars receive red bounding boxes; all others receive blue bounding boxes.

Traffic Signal Detection (Step 2)

A three-mode tiered scan tests RGB, BGR, and Greyscale versions of the image at $\text{imgsz}=1280$ and $\text{conf}=0.03$. If no signals are found, a deep horizontal strip scan divides the image into 10 strips, resizes each to 640×640 , runs inference, and maps coordinates back to original space.

People Ensemble (Step 3)

Only triggered if traffic signals were detected. All five non-car-expert models run across three colour modes, and their bounding boxes are aggregated. OpenCV DNN NMSBoxes with IOU threshold 0.85 provides the final de-duplicated pedestrian count.

GUI Features

- Image uploader with side-by-side annotated output and metrics panel.
- Metrics: Total Cars, Blue Cars, Pedestrians (if applicable), Scene Type.
- Pedestrian count is conditionally hidden for non-traffic-signal scenes with an explanatory warning.

Project 3: Long Hair Detection with Conditional Gender

Problem Statement

Detect long hair in a face image and reclassify gender based on hair length for subjects aged 20-30: long-haired individuals in this age range are labelled female, short-haired ones are labelled male, regardless of biological gender. Subjects outside the 20-30 range receive standard gender prediction. Age group, exact age estimate, hair verdict, and final gender label are all reported.

Dataset

Dataset	CelebAMask-HQ (Kaggle: ipythonx/celebamaskhq)
Size	5,000 high-resolution celebrity face images
Masks Used	Hair segmentation masks from CelebAMask-HQ-mask-anno folder
Format	JPEG images + per-image mask files (prefix: {img_id}_hair.png)

Model Stack

Model	Role	Architecture
SegFormer face-parsing	Pixel-level face part segmentation (class 13 = hair)	SegformerForSemanticSegmentation
Age Classifier	Estimates age bucket (0-2, 3-9, ..., 70+)	HuggingFace image classification pipeline
Gender Classifier	Predicts biological gender	HuggingFace image classification pipeline
MediaPipe FaceLandmarker	Locates chin landmark for hair threshold	MediaPipe Tasks vision API
MediaPipe FaceDetector	Crops face region for downstream models	MediaPipe Tasks vision API

Technical Details

Age Estimation

The age pipeline returns per-bucket probabilities across 8 age groups. An exact age estimate is computed as a weighted sum using bucket midpoints (BUCKET_MAP: 0-2 -> 1, 3-9 -> 6, ..., 70+ -> 75). The highest-probability bucket is reported as the age group label.

Hair Length Detection

SegFormer outputs a logit map which is bilinearly upsampled to the original image resolution. Class 13 (hair) is isolated, morphologically opened to remove noise, and the lowest y-coordinate of any hair pixel

is compared against the chin y-coordinate from the face landmarker. If the hair extends more than 20 pixels below the chin, the verdict is LONG HAIR; otherwise SHORT HAIR / BALD.

Conditional Gender Logic

For subjects with estimated age in [20, 30]: long hair -> female, short hair -> male. Outside this range, the model's gender prediction is returned unchanged. This logic is applied after all individual model outputs are collected.

GUI Features

- Two-column layout: image preview on the left, analysis results on the right.
- Results include numerical age estimate, age group label, hair verdict, and final gender inference.
- All five models loaded once with @st.cache_resource for session-level caching.

Project 4: Nationality & Attribute Identification

Problem Statement

Identify a person's nationality from an image (classified as Indian, American, African, or Other) and detect their emotion. Apply conditional attribute detection: Indians -> age + dress colour + emotion; Americans -> age + emotion; Africans -> dress colour + emotion; Others -> nationality + emotion only.

Dataset

Dataset	UTKFace (Kaggle: jangedoo/utkface-new)
Size	5,000 randomly sampled face images
Filename Format	age_gender_race_timestamp.jpg (race: 0=White, 1=Black, 2=Asian, 3=Indian, 4=Other)
Race Mapping	White -> Americans, Black -> Africans, Indian -> Indians, Asian/Other -> Others

Model Stack

Model	Role	Architecture
YOLOv8n (COCO)	Person detection — locates and crops the subject	YOLO ultralytics
YOLO11x (Custom)	Nationality classification (4 classes)	YOLO classification head
Emotion Pipeline	Facial emotion recognition	HuggingFace image classification
Keras Age Model	Regression-based age prediction	Custom CNN (best_model.h5)

Technical Details

Person Crop

YOLOv8n detects all persons; the highest-confidence detection is selected. The crop is used for nationality and emotion inference. A lower sub-crop (bottom 60% of the person bounding box) is used for dress colour analysis.

Dress Colour Analysis

The clothing crop is resized to 50x50 and flattened to an Nx3 matrix. KMeans clustering (k=1, n_init=5) finds the dominant colour. The RGB centroid is matched to the nearest CSS3 named colour by minimising Euclidean distance in RGB space across all 147 CSS3 colour names via the webcolors library.

Conditional Attribute Logic

A dictionary-based dispatch system selects which additional attributes to compute based on the mapped nationality string. The results dictionary is built dynamically, so the output table only contains columns relevant to the detected nationality.

GUI Features

- Full-width image preview at 1024x1024 pixels.
- Results displayed as a structured Pandas DataFrame table.
- Cache-clearing logic on new file upload (session_state tracking by filename + size).
- Drive folder ID stored in Streamlit secrets for secure deployment.

Project 5: Senior Citizen Video Analysis

Problem Statement

Build a model that processes video files or real-time webcam feeds (simulating a mall or store environment) to detect multiple persons simultaneously, predict their age and gender, flag individuals over 60 as senior citizens, and log all detections with timestamps to a CSV/Excel file.

Dataset

Dataset	UTKFace (Kaggle: jangedoo/utkface-new) — same source as Project 4
Size	5,000 randomly sampled images for model training
Splits	70% train / 15% test / 15% validation
Labels	age (integer), gender (0=Male, 1=Female) parsed from filenames

Model Stack

Model	Role
YOLOv8n (COCO)	Multi-person detection in each video frame
Keras Age Model (best_model.h5)	Per-person age regression (224x224 input, normalised)
HuggingFace Emotion Pipeline	Facial emotion recognition per crop
HuggingFace Gender Pipeline	Gender classification per crop

Technical Details

Video Processing Mode

OpenCV VideoCapture reads the uploaded video file. Five evenly spaced frames are selected using numpy linspace over the total frame count. For each selected frame, YOLOv8n extracts person bounding boxes, and the age, emotion, and gender models process each crop independently. Processed frames are stored in st.session_state to avoid reprocessing on re-render.

Senior Citizen Logic

After prediction, any person with predicted age > 60 is flagged. The DataFrame result is displayed per-frame with ID, Age, Emotion, and Gender columns. The system is designed so that in a production CSV logging scenario, timestamp, age, gender, and senior citizen flag would be appended per detection.

Live Webcam Mode

st.camera_input captures a single snapshot from the browser camera. The captured JPEG is decoded via OpenCV and passed through the same YOLOv8n -> crop -> predict pipeline. Results are shown alongside the annotated image with bounding boxes and ID labels (blue filled rectangles with white text).

Signal Handling

A custom signal handler patches Python's `signal.signal` to suppress `ValueError` exceptions that arise in multi-threaded Streamlit environments when `SIGTERM` or `SIGINT` cannot be registered on non-main threads. This ensures stability in cloud deployments.

GUI Features

- Radio button to switch between File Upload and Live Webcam modes.
- Dropdown to select which of the 5 extracted frames to inspect.
- Side-by-side annotated frame view and per-person results DataFrame.
- No-face-detected warning for empty camera snapshots.

Project 6: ASL Sign Language Recognition

Problem Statement

Train a machine learning model to recognise American Sign Language (ASL) hand signs for both individual alphabets and a set of chosen words. The model should be operable during a defined time window (6 PM to 10 PM). The application must support image upload, video file analysis, and real-time webcam classification.

Dataset

Dataset	American Sign Language Dataset (Kaggle: kapillondhe/american-sign-language)
Size	~5,000 images total — 179 images per class for 28 classes (A-Z + Space + Nothing)
Splits	80% train / 20% validation
Format	Class-organised folder structure (ASL_Dataset/Train/{class_name}/)

Models

Model	File	Task
Alphabet Classifier	best_image_alphabet.pt	YOLO classification — identifies single letter signs from still images
Word Classifier	best_video_words.pt	YOLO classification — identifies whole ASL word signs from video frames or live feed

Both models are YOLO classification models trained on ASL data. They are downloaded from Google Drive at startup via gdown and cached with @st.cache_resource.

Technical Details

Alphabet Recognition (Image)

A static image is uploaded, passed directly to the alphabet YOLO model, and the top-1 prediction is returned. The YOLO classification head outputs class probabilities; `res[0].probs.top1` gives the index and `res[0].names` maps it to a label.

Word Recognition from Video

A sliding-window majority-vote system (`WINDOW_SIZE=12`, `VOTE_THRESHOLD=8`) is applied frame-by-frame. A word is confirmed only if it appears in at least 8 of the last 12 frames and is different from the last confirmed word. Space and Nothing classes are filtered from the output. A Streamlit progress bar tracks the frame processing status. Detected words are concatenated into a final sentence.

Live Webcam (streamlit-webrtc)

The ASLTransformer class inherits from VideoTransformerBase and processes each WebRTC video frame in real time. A rolling window of the last 10 predictions is maintained; the most common label in the window is displayed as an overlay on the frame using OpenCV putText.

Application Tabs

Tab	Mode	Model Used
Alphabet (Image)	Upload static image	best_image_alphabet.pt
Word (Image)	Upload static image	best_video_words.pt
Word (Video)	Upload video file, background analysis	best_video_words.pt
Live Webcam	Real-time WebRTC stream	best_video_words.pt

Common Technical Stack & Architecture Patterns

Technology Stack

Category	Library / Tool	Usage
UI Framework	Streamlit	All 6 applications — file upload, display, widgets, session state
Object Detection	Ultralytics YOLOv8 / YOLO11	Projects 2, 4, 5, 6 — person, car, sign detection
Audio ML	Wav2Vec2 (transformers), librosa	Project 1 — voice feature extraction
Computer Vision	OpenCV (cv2)	Projects 2, 3, 5 — image processing, annotation
Deep Learning	PyTorch	Project 1 — custom model heads, inference
Transformer Pipelines	HuggingFace transformers	Projects 1, 3, 4, 5 — classification pipelines
Image Segmentation	SegFormer (transformers)	Project 3 — face parsing / hair segmentation
Face Landmarks	MediaPipe Tasks	Project 3 — face detection, landmark extraction
Keras	TensorFlow / Keras	Projects 4, 5 — age regression model (best_model.h5)
Colour Analysis	webcolors, KMeans (scikit-learn)	Project 4 — dominant colour identification
Model Download	gdown	All 6 — runtime download from Google Drive
Data Handling	pandas, numpy	All 6 — dataframe operations, numerical processing
Live Streaming	streamlit-webrtc	Projects 5, 6 — browser webcam integration

Common Architectural Patterns

1. Runtime Model Download

All models are stored on Google Drive and downloaded at first launch using `gdown.download_folder` or `gdown.download`. A local path existence check prevents re-downloading on subsequent runs. This keeps repository size minimal and enables easy model version updates.

2. Streamlit Caching

Every model loading function is decorated with `@st.cache_resource`, which persists loaded model objects in memory across user interactions within a session. This is critical for large transformer models (Wav2Vec2, SegFormer) that take significant time to load.

3. Conditional Inference Pipelines

A defining architectural choice across all projects is the use of sequential gating: earlier, cheaper models (gender check, person detection, scene classification) determine whether more expensive downstream models are invoked. This reduces unnecessary computation and enforces problem-specific business rules cleanly in code.

4. Session State Management

Projects 2 and 5 use `st.session_state` to cache processed results, preventing reprocessing when the user interacts with dropdowns or other widgets after initial inference. A file identity key (name + size) triggers a cache clear and re-run when a new file is uploaded.

Data Preparation & Training Pipelines

Each project follows a structured notebook-based pipeline from dataset acquisition to model-ready splits. The common phases are described below.

Phase 1: Data Acquisition

All datasets are sourced from Kaggle using kagglehub or kaggle CLI, and transferred to Google Colab's /content/ directory for processing. Google Drive serves as the persistent storage backend for prepared datasets and trained model weights.

Project	Dataset	Source
Voice Age/Emotion	Common Voice v4	Kaggle (vedant2022/common-voice-dataset-version-4)
Car Colour	Indian Driving Dataset	Kaggle (redzapdos123/indian-driving-dataset-detections-yolov11)
Long Hair	CelebAMask-HQ	Kaggle (ipythonx/celebamaskhq)
Nationality	UTKFace	Kaggle (jangedoo/utkface-new)
Senior Citizen	UTKFace	Kaggle (jangedoo/utkface-new)
Sign Language	ASL Dataset	Kaggle (kapillondhe/american-sign-language)

Phase 2: Data Preparation

Each notebook contains detailed preparation steps:

- Balanced sampling: audio clips per age group (Project 1), nationality-stratified samples (Projects 4/5), ASL class-limited copies (Project 6)
- CSV manifest generation: metadata files linking image/audio paths to labels, enabling reproducible training
- Train/Test/Val splitting: typically 70/15/15 for image projects using sklearn train_test_split with random_state=42 for reproducibility
- Organised folder structures: split-specific subdirectories mirroring YOLO or Keras expected formats

Phase 3: Model Training

Image classification and detection models are trained using the Ultralytics YOLO framework (YOLOv8/YOLO11) in Google Colab with GPU acceleration. Keras/TensorFlow models are trained with early stopping and best-weight checkpointing (best_model.h5). Pre-trained transformer models from HuggingFace are used as-is without fine-tuning for emotion, gender, and age bucket classification.

Phase 4: Model Persistence

Trained model weights (.pt for YOLO, .h5 for Keras, saved_model directories for HuggingFace) are uploaded to Google Drive. Runtime download is handled by gdown with folder or file-level granularity. Drive folder IDs are configured either directly in the app code or via Streamlit secrets for secure production deployments.

Results & Evaluation Summary

Each project was evaluated on its logical correctness, GUI functionality, and model integration. The following table summarises the key evaluation dimensions:

Project	Core Logic	GUI	Key Constraint Implemented
Voice Age & Emotion	Tiered audio pipeline	File upload + audio player	Female voice rejection; emotion only for age > 60
Car Colour & Traffic	6-model ensemble + IoU dedup	Image + metrics panel	People count hidden for non-signal scenes
Long Hair	Hair segmentation + chin landmark	Two-column analysis view	Hair-based gender override only for age 20-30
Nationality ID	YOLO11 classification + dispatch	Full image + table output	Conditional attributes per nationality
Senior Citizen	Frame sampling + multi-face	Frame selector dropdown	Senior flag for age > 60; webcam snapshot mode
Sign Language	Sliding-window majority vote	4-tab interface	Video word detection with noise filtering

Strengths

- Consistent architectural patterns across all six projects ensure maintainability and reusability.
- Dynamic class ID discovery in Project 2 makes the ensemble future-proof against YOLO model updates.
- Conditional inference gates reduce compute overhead and enforce domain logic cleanly.
- Session-state caching prevents redundant reprocessing in interactive UIs.
- All models are decoupled from the application code through Google Drive, enabling independent updates.

Limitations & Future Work

- Age regression accuracy is sensitive to audio/image quality; confidence intervals are not currently displayed.
- The blue car detector uses a simple channel-ratio rule; a learned colour classifier would improve accuracy under varied lighting.
- The sign language time-restriction feature (6 PM - 10 PM) is mentioned in the problem statement but not implemented in the deployed app; it would require server-side time checking with timezone support.
- Video processing in Project 5 samples only 5 frames; a full-video temporal analysis would improve detection coverage.

- Real-time webcam streaming (streamlit-webrtc) requires STUN/TURN server configuration for production deployment.

Conclusion

This suite of six applications demonstrates a breadth of applied machine learning capabilities — spanning computer vision, natural language audio processing, object detection, semantic segmentation, and classification — all unified through a common deployment pattern on Streamlit.

Each project translates a real-world problem statement into a working end-to-end system with a clear data pipeline, trained or integrated models, conditional inference logic, and an accessible graphical user interface. The consistent use of Google Drive for model storage, gdown for runtime loading, and Streamlit's caching infrastructure reflects a production-aware engineering approach.

The projects collectively showcase: multi-model ensembling (Project 2), transformer-based feature extraction (Projects 1 and 3), conditional attribute routing (Projects 3 and 4), temporal video analysis (Projects 5 and 6), and real-time streaming inference (Projects 5 and 6). Together they form a cohesive portfolio of AI application development from data to deployment.

End of Report